

LLM-Assisted Multi-Agent Stock Decision Support System

Final Course Project Presentation

Mo Di (2021030019)

Representation and Reasoning: Foundations of Large Models

Final Presentation

Problem and Goal

- Build an applied prototype for daily-frequency simulated stock decision support.
- Support three tasks: single-stock analysis, candidate screening, and portfolio rebalancing.
- Avoid a black-box “ask the LLM for a stock pick” workflow.
- Produce reproducible outputs, explicit risk warnings, and benchmarkable behavior.

System Workflow

Data Layer	Agent Layer	Decision Layer	Evaluation Layer
Prices, news, fundamentals, constraints	Market, sentiment, fundamental, risk agents	Decision synthesis plus optional OpenRouter reviewer	Fixed cases, baselines, backtest, dashboard

Design principle

The LLM sees structured intermediate evidence and reviews the decision; deterministic fallback keeps the benchmark reproducible.

Signal agents

- MarketInformationAgent
- NewsSentimentAgent
- FundamentalAgent

Control agent

- RiskManagerAgent checks volatility, drawdown, concentration, cash, and trade limits.

Decision agents

- DecisionAgent combines weighted evidence.
- LLMDecisionAdvisor optionally adjusts the result through OpenRouter JSON review.

Output

Action, score, confidence, rationale, risk warnings, target weights, and trade deltas.

Data and Benchmark

- Offline processed dataset: 12 large-cap tickers, 60 trading days.
- Inputs: OHLCV price history, sample news events, fundamentals, and portfolio constraints.
- Final benchmark: 15 fixed cases.

Task	Cases	Purpose
Single-stock analysis	10	action, rationale, risk awareness
Candidate screening	2	ranked stock pool selection
Rebalancing	3	portfolio constraints and trade deltas

Evaluation Results

Metric	Result
Unit tests	6/6 passing
Final benchmark cases	15/15 passing
Multi-agent risk warnings	16
Equal-weight cumulative return	4.77%
Equal-weight max drawdown	-1.37%
Equal-weight Sharpe ratio	2.44

Interpretation

The benchmark evaluates structure, constraints, and reproducibility. It is not a claim of live trading profitability.

Demo Plan

① Run the final benchmark:

```
.\.venv\Scripts\python.exe scripts\run_demo.py --task all --llm off
```

② Open generated outputs:

- reports/final_benchmark_results.md
- reports/final_dashboard.html

③ Optional LLM mode:

```
$env:OPENROUTER_API_KEY="..."
```

```
.\.venv\Scripts\python.exe scripts\run_demo.py --task final --llm required
```

- Runnable Python package under `src/stock_agent/`.
- CLI entry point: `scripts/run_demo.py`.
- Deterministic data generator: `scripts/generate_final_data.py`.
- Automated tests under `tests/`.
- Reports: Markdown summary and static HTML dashboard.
- Final written report and this presentation deck.

Limitations and Next Steps

- Current data is deterministic and offline; live market/news feeds are future work.
- Scoring rules are transparent but simple; calibrated statistical models could improve them.
- The backtest is auxiliary and equal-weight, not an optimized trading strategy.
- Next steps: real OpenRouter run record, richer demo video, broader stock universe, and more realistic transaction-cost modeling.

Takeaway

Final outcome

The project has moved from a minimal prototype to a complete applied-track deliverable: runnable system, fixed benchmark, baselines, tests, generated reports, final report, and presentation slides.